

Introduction to Memory Management & Virtual Memory Fundamentals

1. What is Memory Management?

Definition and Purpose

Memory management is the function of an operating system (OS) that is responsible for **handling and organizing computer memory**. It determines how memory is allocated to processes, how it is used, and how it is reclaimed when no longer needed.

The primary purposes are:

- Ensuring each program has enough memory to run
- Protecting memory spaces from interfering with one another
- Optimizing the use of limited physical memory
- Allowing multiple programs to run simultaneously

In short, memory management ensures **efficient, safe, and flexible use of system memory**.

Role of the Operating System (OS)

The OS acts as the **manager** of all memory resources. Its responsibilities include:

- Allocating memory to processes when they start
- Tracking which parts of memory are in use
- Deciding which data should stay in RAM and which can be stored on disk
- Reclaiming memory when processes finish
- Enforcing permissions to keep programs isolated

The OS works with hardware components—especially the **Memory Management Unit (MMU)**—to perform address translation and ensure memory protection.

Goals of Memory Management

1. Efficiency

Memory is a limited resource. The OS must use it efficiently so that:

- Multiple programs can run at once
- System performance is maximized
- Minimal memory is wasted

Efficient memory management improves CPU utilization and overall system throughput.

2. Protection

Each process should have its own private memory segment. Memory protection ensures:

- One process cannot accidentally or intentionally modify another's memory
- System stability is maintained

- Security vulnerabilities are minimized

Protection is enforced by hardware (MMU) and by creating “boundaries” between processes.

3. Transparency

- To programmers, memory should appear as:
 - Large
 - Continuous
 - Easy to use
- They should not worry about:
 - Where the program is physically stored
 - Whether memory is fragmented
 - How data moves between RAM and disk
- Virtual memory provides this abstraction.

4. Flexibility

The system should handle many types of memory needs:

- At runtime, processes request varying memory sizes
- Memory can grow or shrink
- Shared memory regions may be used between processes

The OS must be able to adapt dynamically.

2. Physical vs. Logical Address Spaces

Physical Address

A **physical address** refers to the actual location in **actual RAM hardware**.

Examples:

- Address 0xA000
- Address 0x1F3C2

The CPU reads/writes data stored at these physical memory locations.

Logical (Virtual) Address

A **logical address**, also called a **virtual address**, is generated by the CPU when a program is executed. Programs work only with virtual addresses—not physical ones.

Example:

A program might think its variable is at address 0x1000, even if it is actually stored at physical address 0x3A450.

Why Different Address Spaces?

- Allows each process to believe it has access to a large, continuous block of memory
- Provides isolation between processes
- Makes it easier to relocate programs in memory

- Supports virtual memory (RAM + disk)

Memory Management Unit (MMU)

The **MMU** is a hardware component that:

- Translates virtual addresses to physical addresses
- Enforces access permissions
- Works together with page tables

This translation happens **every time a memory address is used**.

Address Translation Basics

When the CPU generates a virtual address:

1. It sends the address to the MMU.
2. The MMU looks up the translation in the page table.
3. The MMU outputs the physical address.
4. The RAM is accessed at that physical location.

This process is fast, and often accelerated by hardware caches like the **TLB (Translation Lookaside Buffer)**.

3. Virtual Memory: Concept & Motivation

Virtual memory is an OS mechanism that gives processes the illusion of having more memory than is physically available.

Why Virtual Memory Is Needed

1. Limited Physical RAM

Computers have finite RAM.

Virtual memory allows programs larger than the available RAM to run by using part of the disk as extension.

2. Large Program Support

Programs often need much more memory than available at once.

Virtual memory loads only the parts actually needed into RAM.

3. Isolation Between Processes

Each process has its own private virtual address space.

No process can access another's memory unless permitted, improving security.

Benefits of Virtual Memory

1. Simplified Programming Model

Programs behave as if:

- They have access to a large, continuous memory space

- Memory always starts at address zero

The OS handles mapping and fragmentation behind the scenes.

2. Security and Protection

Each process gets its own virtual space.

Memory protection prevents:

- Crashes caused by pointer errors
- Security vulnerabilities where one process reads another's data

3. Multitasking Efficiency

Virtual memory enables multiple programs to:

- Run at the same time
- Share memory efficiently
- Be swapped in/out depending on usage

This increases system responsiveness and allows more processes to run.

4. Basic Virtual Memory Architecture

Virtual Address Space

A virtual address space is the set of addresses a program can use.

Typically ranges:

- 32-bit → 4 GB virtual space per process
- 64-bit → up to terabytes of virtual space

Each program believes it has its own full address space.

Physical Address Space

This is the actual RAM available on the machine (e.g., 4 GB, 8 GB, 16 GB).

The OS maps **virtual addresses** to **physical addresses** based on availability.

Address Translation via Page Tables

A **page table** is a data structure used by the OS to map virtual pages to physical frames.

For each virtual page, the page table stores:

- Whether the page is in memory
- If so, which physical frame it resides in
- Access permissions

The MMU uses this table to translate virtual addresses continuously.

Role of the MMU and Hardware Support

The MMU:

- Performs fast address translation
- Enforces memory protection
- Supports virtual memory through paging
- Can detect page faults and notify the OS

Hardware support is essential for performance and reliability.

5. Paging Basics

Paging is a method of virtual memory that divides memory into **fixed-size blocks**.

Pages and Frames

- **Page:** A fixed-size block of virtual memory (e.g., 4 KB)
- **Frame:** A fixed-size block of physical memory (same size as pages)

Paging eliminates external fragmentation, because all blocks are uniform in size.

How Pages Are Mapped to Frames

The OS maintains a **page table** for each process.

Example mapping:

- Virtual Page 0 → Physical Frame 4
- Virtual Page 1 → Physical Frame 9
- Virtual Page 2 → On disk (not in RAM)
- Virtual Page 3 → Physical Frame 2

If a page is needed but not in RAM, a **page fault** occurs.

Page Table Entry (PTE) Components

Each PTE contains important information:

1. Frame Number

Indicates which physical frame the virtual page is stored in.
If the page is not in RAM, this field might be empty.

2. Valid/Invalid Bit

- **Valid = 1** → Page is present in RAM
- **Invalid = 0** → Page not in physical memory (may be on disk)

Invalid entries cause page faults when accessed.

3. Protection Bits

Define what kind of operations are allowed:

- Read
- Write
- Execute

These bits prevent unauthorized access and help enforce process isolation.

Advanced Paging & Virtual Memory Mechanisms

This lecture expands on paging, address translation, performance optimizations, and key virtual memory mechanisms used in modern operating systems.

1. Multi-Level Paging

1.1 Why Multi-Level Paging Is Needed

A single-level page table can become extremely large.

Example:

- 32-bit system
- 4 KB pages → each page offset requires 12 bits
- Remaining 20 bits for page numbers
- Total pages per process: $2^{20} = 1,048,576$ pages

If each Page Table Entry (PTE) is 4 bytes:

- Single page table size = 4 MB per process
- With many processes, memory is wasted.

Solution: **Break the page table into multiple levels** so that only required sections are stored.

1.2 Two-Level Paging

Virtual address is divided into:

- Outer (page directory) index
- Inner (page table) index
- Offset within page

Advantages:

- Only page-table sections that are needed are allocated in RAM
- Saves memory when processes use sparse address spaces

1.3 Hierarchical or Multi-Level Paging

Modern 64-bit systems may use:

- 4-level
- 5-level

- or even more

Example (x86-64):

- PML4
- PDPT
- Page Directory
- Page Table
- Offset

These levels drastically reduce memory overhead by allocating only necessary structures.

2. Translation Lookaside Buffer (TLB)

2.1 What Is the TLB?

A **Translation Lookaside Buffer (TLB)** is a specialized CPU cache storing **recent virtual-to-physical address translations**.

Why needed?

Because multi-level page tables require multiple memory accesses.

TLB reduces this to **one fast lookup**.

2.2 TLB Hit vs. TLB Miss

- **TLB Hit:** MMU finds the translation in TLB → fast
- **TLB Miss:** Must walk page tables → slower

Processors often have:

- Separate instruction TLB (ITLB)
- Separate data TLB (DTLB)
- Unified TLB at last level

2.3 TLB Replacement Policies

When TLB is full, entries must be replaced. Common strategies:

- LRU (Least Recently Used)
- Random
- FIFO

TLB performance is crucial for system speed.

3. Page Faults

3.1 What Is a Page Fault?

Occurs when a program accesses a **virtual page not in physical memory**.

Types:

- **Minor fault:** Page exists in memory but not mapped (e.g., copy-on-write).
- **Major fault:** Page must be loaded from disk → expensive.

3.2 Steps in Handling a Page Fault

1. CPU detects missing page → traps to OS
2. OS checks validity of requested address
3. If valid:
 - Locate the page on disk (swap or executable file)
 - Select a free frame (or use replacement algorithm)
 - Load page into frame
4. Update page table entry
5. Resume the process

This process can take **microseconds to milliseconds**, depending on whether disk I/O is needed.

3.3 Impact on Performance

Frequent page faults slow down programs dramatically.
Reducing page faults is a key purpose of paging algorithms.

4. Demand Paging

4.1 Definition

Demand paging loads pages **only when they are needed**, instead of loading the entire program into memory at startup.

Advantages:

- Faster startup time
- Reduced memory usage
- Allows large programs to run even with limited RAM

4.2 Lazy Allocation

Memory is allocated only when accessed, not when requested.

Example:

When a program calls `malloc(1GB)`, pages are not allocated until actually touched.

4.3 Handling First Access

The first access to a page triggers a **page fault**, causing the OS to allocate and map the page.

5. Copy-on-Write (CoW)

5.1 Concept

Copy-on-Write allows two processes to **share the same physical pages** until one of them tries to write.

Used heavily in:

- `fork()` system calls
- Virtual machine cloning
- Containers

5.2 How It Works

1. Parent and child share pages in read-only mode
2. When either tries to write:
 - Page fault occurs
 - OS creates a private copy for the writer
3. Other process retains shared original page

Benefits:

- Saves significant memory
- Speeds up process creation

6. Memory Protection

6.1 Why Protection Is Needed

To prevent processes from:

- Overwriting each other's data
- Accessing unauthorized memory
- Crashing the entire system

6.2 Types of Memory Protection

1. Read, Write, Execute Permissions

Each page may be marked:

- Read-only
- Write-only
- Read-write
- Execute-only

2. Privilege Levels

Certain memory areas (e.g., kernel memory) are protected from user-mode access.

3. Segmentation (historical but still used in some systems)

Segments:

- Code
- Data
- Stack

Each has protection and bounds.

Segmentation is often combined with paging for flexibility and safety.

Paging Algorithms, Thrashing, and Modern Memory Management

This lecture covers page replacement algorithms, frame allocation, thrashing, and modern virtual memory techniques used in operating systems today.

1. Page Replacement Algorithms

When a page fault occurs and RAM is full, the OS must choose **which existing page to remove** from memory to make space. This is handled by **page replacement algorithms**.

Goals of replacement algorithms:

- Reduce page faults
- Improve performance
- Maintain locality of reference

Below are key algorithms used historically and in modern OS design.

1.1 FIFO (First-In, First-Out)

Concept

- The **oldest** loaded page is replaced first.
- Pages are arranged in a queue; the page at the front is removed.

Advantages

- Simple to implement
- Minimal overhead

Disadvantages

- Poor performance in many cases
- Can replace heavily used pages
- **Belady's Anomaly**: Increasing frame count may **increase** page faults

1.2 OPT (Optimal Algorithm)

Concept

Replaces the page that **will not be used for the longest time in the future**.

Advantages

- Produces the lowest possible number of page faults
- Serves as a theoretical benchmark

Disadvantages

- Cannot be implemented in real systems (requires future knowledge)
- Only useful for analysis and comparison

1.3 LRU (Least Recently Used)

Concept

Replaces the page that has not been used for the **longest time in the past**.

Advantages

- Approximates OPT based on past behavior
- Works well in practice because programs exhibit **temporal locality**
- Widely used

Disadvantages

- Expensive to implement perfectly (requires time-stamping)
- Hardware support or approximate algorithms are often needed

1.4 Clock / Second-Chance Algorithm

Concept

A practical approximation of LRU.

Process:

1. Pages arranged in a circular list (like a clock)
2. Each page has a **reference bit**
3. On replacement:
 - If reference bit = 0 → replace the page
 - If reference bit = 1 → give a second chance (set bit to 0 and move on)

Advantages

- Efficient

- Good balance between performance and simplicity
- Common in real-world systems

Disadvantages

- Not as precise as LRU
- May require multiple scans if many bits are set

2. Frame Allocation Policies

OS must decide **how many frames to allocate to each process**.

2.1 Equal Allocation

Each process receives the same number of frames.

- Example:
 - 4 processes
 - 40 frames
 - Each gets 10 frames
- **Pros:**
 - Simple
- **Cons:**
 - Large processes suffer
 - Small processes waste memory

2.2 Proportional Allocation

Frames are allocated based on each process's size.

- Example:
 - Process A: 50 MB
 - Process B: 100 MB
 - B would get twice as many frames as A.
- **Pros:**
 - Fair
 - Accounts for memory needs
- **Cons:**
 - Still may under-allocate memory to large or active processes

2.3 Local vs. Global Replacement Policies

Local Replacement

- Process can replace only the pages that belong to **itself**
- More predictable
- Avoids one process affecting others

Global Replacement

- Page can be selected from **any process**
- Allows system-wide optimization
- But one process may steal frames from others (can cause thrashing)

3. Thrashing

3.1 What Is Thrashing?

Thrashing occurs when a system spends **more time handling page faults than executing instructions**.

Symptoms:

- High disk activity
- Low CPU utilization
- Programs slow to a crawl

Thrashing happens when the working set of a process cannot fit into available memory.

3.2 Causes of Thrashing

- Too many active processes
- Insufficient frames allocated
- Poor locality of reference
- Excessive global page replacement

3.3 How to Detect Thrashing

- A rapid increase in page faults
- CPU utilization drops
- Memory pressure indicators rise

Operating systems often track page-fault frequency (PFF) to identify thrashing.

3.4 Methods to Reduce Thrashing

1. Working Set Model

The OS monitors the “working set”:

- Recently used pages
- Typically measured within a sliding window (Δ time)

If a process’s working set does not fit into available frames, the OS may:

- Allocate more frames
- Suspend the process
- Swap out entire processes

2. Page Fault Frequency (PFF) Control

If page faults:

- Increase → give more frames
- Decrease → take frames away

Maintains stable system performance.

3. Reduce Degree of Multiprogramming

Suspending or killing processes to free memory.

4. Use Better Replacement Algorithms

LRU or Clock can reduce thrashing dramatically compared to naive algorithms like FIFO.

4. Swapping and Swap Space

4.1 What Is Swapping?

Swapping moves entire processes in and out of RAM to free memory.

4.2 Where Is Swap Stored?

- Swap partitions
- Swap files
- Virtual memory extensions

4.3 Performance Consideration

Swap operations involve disk I/O → **slow**
Excessive swapping contributes to thrashing.

5. Modern Memory Management Techniques

5.1 Memory-Mapped Files

Allows file contents to be mapped directly into virtual memory.

Advantages:

- Fast I/O
- No need for block-by-block reading
- Efficient file sharing between processes

Used in:

- Databases
- Shared libraries
- High-performance applications

5.2 Huge Pages (Large Pages)

Systems often support:

- 2 MB pages
- 1 GB pages

Advantages:

- Reduces page table size
- Reduces TLB misses
- Boosts performance for large-memory applications

Common in:

- Databases
- Machine learning workloads
- Virtual machines

5.3 NUMA (Non-Uniform Memory Access) Awareness

Modern multi-core systems have memory regions with different access speeds.

NUMA-aware memory management:

- Allocates memory close to the CPU accessing it
- Improves performance
- Reduces latency

5.4 Hardware Virtualization (VT-x, AMD-V)

Hardware accelerates:

- Virtual memory translation
- Page table switching
- TLB tagging (ASIDs – Address Space Identifiers)

Improves performance for:

- Virtual machines
- Containers
- Sandboxed environments